

# VI.6 – Analyse de la Dynamique du Projet

Coordinateur de projets informatiques · Projet d'étude fil rouge certifiant 2025-2026

SUP DE VINCI 4  
Exemplaire candidat

Rôle : Développeur Front-End

Projet : Althea Systems – E-commerce médical B2B

Calendrier : Livrables finaux + 2 semaines

Ce document constitue une réflexion personnelle sur mon expérience au sein du projet **Althea Systems**, une plateforme e-commerce B2B dédiée à la distribution de matériel médical de haute précision. Mon rôle au sein de l'équipe a été celui de **développeur front-end**, en charge de la conception et de l'implémentation de l'interface utilisateur sous **Angular 21**, ainsi que de l'intégration de fonctionnalités transverses : chatbot IA, module de traduction automatique, connexion aux APIs Stripe et Cloudinary.

## 1 · Réflexion personnelle sur les défis rencontrés

### 1.1 – LES MOMENTS DIFFICILES

#### Intégration du chatbot (Ollama)

Technique · Complexité élevée

Le chatbot repose sur une instance **Ollama** déployée via Docker, interrogée par le backend ASP.NET Core via un client dédié. Côté front, il fallait gérer un **flux de réponse en streaming** (Server-Sent Events), concevoir une interface conversationnelle réactive et maintenir une expérience fluide malgré la latence du modèle.

Les premières tentatives ont produit des comportements erratiques : messages tronqués, états de chargement bloqués, scroll automatique défaillant. Le débogage était rendu difficile par le caractère non-déterministe des réponses — les tests unitaires classiques s'avéraient insuffisants ; il fallait tester en conditions réelles, en boucle.

### Intégration de LibreTranslate

Technique · Fiabilité service

Le service tourne en conteneur Docker et expose une API REST. L'enjeu côté front était d'orchestrer les appels de traduction à la volée sans dégrader l'expérience : éviter les **flashes** de contenu non traduit, gérer les délais de démarrage du conteneur, et traiter proprement les cas d'indisponibilité du service.

Il a fallu mettre en place une stratégie de **fallback** — afficher la langue source en cas d'échec plutôt que de casser l'interface — et aligner la gestion des erreurs avec le backend pour éviter que les exceptions remontent à l'utilisateur.

### Gestion du temps et des demandes en fin de sprint

Organisationnel

Le front-end étant la couche visible du projet, il génère naturellement des demandes de modifications fréquentes en fin de sprint : ajustements UI, corrections responsive, alignements visuels. Ces retouches s'accumulent et empiètent sur le travail de fond. Trouver le bon équilibre entre réactivité aux retours et avancement des tâches structurantes a été un effort constant.

## 1.2 – COMMENT CES DIFFICULTÉS ONT ÉTÉ ABORDÉES ET ENSEIGNEMENTS

Pour le chatbot, j'ai adopté une **approche itérative** : construire d'abord une version minimale fonctionnelle (requête-réponse simple, sans streaming), puis introduire la complexité par couches successives. Cela a permis d'isoler précisément la source de chaque problème. Pour le streaming, j'ai étudié l'API des SSE et adapté les composants Angular via des **Observables** RxJS pour piloter l'état de l'interface.

Pour LibreTranslate, la résolution est passée par une meilleure compréhension du cycle de vie des conteneurs Docker et une **communication plus étroite avec l'équipe backend**, afin d'aligner nos stratégies de retry et de timeout en amont plutôt qu'en réaction.

**Principaux enseignements** : l'intégration de services tiers exige de traiter les cas d'échec avec autant de soin que le cas nominal. La communication inter-couche n'est pas une option, c'est une condition de réussite dès que la fonctionnalité traverse plusieurs parties du système.

## 2 · Identification des forces et faiblesses

---

### POINTS FORTS

- **Autonomie technique** — prise en main rapide de technologies nouvelles (SSE, Docker Compose, Angular 21 standalone) sans bloquer l'équipe.
- **Rigueur sur la qualité visuelle** — cohérence avec la charte graphique (palette teal/navy, typographie Cormorant/DM Sans), attention aux états interactifs (hover, focus, loading, error).
- **Écoute et adaptabilité** — intégration des retours d'équipe sur l'ergonomie sans résistance, comme un levier d'amélioration plutôt qu'une remise en question.
- **Communication proactive** — anticipation des contrats d'API avec le backend avant de coder les composants.

### LIMITES RÉVÉLÉES

- **Sous-estimation des intégrations** — tendance initiale à planifier le temps front sans anticiper la complexité des interfaces avec les services externes.
- **Documentation insuffisante** — sous pression, négligence des commentaires sur les workarounds non-évidents, créant des zones d'ombre pour les autres développeurs.
- **Perfectionnisme en fin de livraison** — tendance à vouloir tout peaufiner dans les dernières heures, pas toujours compatible avec les contraintes de planning.

Sur le dernier point, j'ai progressé en apprenant à tester les interfaces d'API dès le début d'une tâche — ce qui décale en amont la détection des problèmes d'intégration — et en me fixant des critères de "done" explicites plutôt que subjectifs.

### 3 · Compétences développées

#### TECHNIQUES

- Angular 21 — standalone components, routing lazy-loaded, intercepteurs HTTP
- RxJS — Observables, gestion d'état asynchrone
- Server-Sent Events pour le streaming IA
- Intégration Stripe, Cloudinary, LibreTranslate
- Tailwind CSS dans un design system contraint
- Docker Compose multi-services
- Tests unitaires Vitest (services Angular)

#### ORGANISATIONNELLES

- Découpage de tâches front indépendantes du backend (mocks, interfaces TypeScript)
- Git en équipe — branches feature, pull requests, résolution de conflits
- Rédaction de documentation technique synthétique
- Gestion des priorités en fin de sprint
- Critères de "done" définis en amont

#### HUMAINES

- Communication régulière avec le backend sur les contrats d'API
- Arbitrage entre perfectionnisme et pragmatisme
- Gestion de la frustration face aux bugs non-reproductibles
- Réception des retours sans résistance défensive
- Travail en équipe sur un périmètre technique large

### 4 · Axes d'amélioration pour un futur projet

#### CE QUE JE CHANGERAIS

- **Phase de cadrage technique renforcée** sur les intégrations externes : définir les contrats d'API, identifier les risques d'indisponibilité, planifier les fallbacks dès le démarrage — pas en réaction.
- **Documentation légère mais systématique** : un commentaire explicatif sur chaque workaround non-évident, un document de décisions techniques partagé. Ce n'est pas une contrainte, c'est une économie de temps à moyen terme.
- **Critères d'acceptation visuels** définis avec l'équipe en amont pour limiter les aller-retours d'UI en fin de sprint.

#### CE QUE JE CONSERVERAIS

- **Approche itérative sur les fonctionnalités complexes** — construire d'abord un squelette fonctionnel, puis enrichir. Cette méthode a systématiquement raccourci le temps de débogage.
- **Collaboration front/backend en continu** — le découplage technique ne signifie pas l'isolement. Synchronisation régulière dès le démarrage d'une fonctionnalité.
- **Exigence de qualité sur tous les états UI** — loading, erreur, vide, disabled. Dans un contexte B2B professionnel, ce niveau de soin est une vraie valeur ajoutée produit.

## Conclusion

---

Ce projet m'a permis de consolider une posture de développeur front-end capable d'aller au-delà du composant isolé pour traiter des problématiques d'intégration systémique. Les difficultés rencontrées — notamment autour du chatbot Ollama et de LibreTranslate — ont été autant d'occasions d'apprentissage concret, ancré dans des contraintes réelles et non simulées.

Je repars avec une meilleure conscience de mes limites, une méthodologie plus rigoureuse pour aborder les intégrations complexes, et la conviction que la communication d'équipe est aussi déterminante que la qualité du code. La prochaine fois, je passerai moins de temps à déboguer des problèmes que j'aurais pu anticiper — et plus de temps à construire.